United International University

Grambandhan

A Smart Rural Investment Platform

Department of Computer Science & Engineering
PMG 4101 – Project Management

Group No.: 6

Submitted By

Sourav Saha	0112330638
MD Jahidul Islam Jony	0112330060
Saptarshi Biswas Supty	0112330221
Partha Pratim Podder	0112331079
Mahin Hasan Upol	0112230520

Submitted To

Syed Samin Sadaf
Lecturer
Department of Computer Science & Engineering
United International University

Contents

1 Project Overview **5**

1.1 Problem Statement 5

1.2 Vision 6

1.3 Scope and Features 6

2 Methodology **7**

2.1 What is Methodology 7

2.2 Waterfall Model 8

2.2.1 Waterfall Model Design 8

2.2.2 Advantages of the Waterfall Model 8

2.2.3 Limitations of the Waterfall Model 9

2.3 Scrum Development Methodology 9

2.3.1 Roles and Responsibilities in Scrum 9

2.3.2 How the Scrum Workflow Operates 9

2.3.3 Advantages of Scrum Development 10

2.3.4 Challenges of Scrum Development 10

2.4 Prototype Methodology 10

2.4.1 Designing a Prototype for Grambandhan 11

2.4.2 Advantages of the Prototype Model 11

2.4.3 Limitations of the Prototype Model 11

2.5 Incremental Model 11

2.5.1 Planning Development Increments 12

2.5.2 Advantages of the Incremental Model 12

2.5.3 Limitations of the Incremental Model 13

2.6 Extreme Programming (XP) Methodology 13

2.6.1 Applying XP Practices to Grambandhan 13

2.6.2 Advantages of Extreme Programming 13

2.6.3 Challenges of Extreme Programming 14

2.7 Our Project Methodology 14

2.8 Methodology Phases 14

3 Project Work Decomposition **16**

3.1 Purpose and Structure of the WBS 16
3.2 Guidelines for Organizing Project Deliverables 16
3.2.1 Complete Scope Coverage 16
3.2.2 Defining Deliverables Clearly 16
3.2.3 Establishing the Main Deliverable Groups 17
3.2.4 Information Required for Each Work Package 17
3.2.5 Avoiding Overlap Between Work Packages 17
3.2.6 Breaking Down Large Tasks 18
3.2.7 Reviewing Work Package Size 18
3.3 Grambandhan Deliverable Breakdown 18

4 Wideband Delphi Estimation **20**
4.1 Wideband Delphi Estimation Technique 20
4.2 Steps in the Wideband Delphi Process 21
4.2.1 Selection of the Estimation Team 21
4.2.2 Estimation Kick-off Meeting 21
4.2.3 Independent Task Estimation 21
4.2.4 Comparison and Discussion of Estimates 22
4.2.5 Consolidation of Work Packages 22
4.2.6 Review of Estimation Results 22
4.2.7 Initial Task Effort Summary 22
4.3 Individual Three-Point Estimation Form 23
4.4 Analysis of the Initial Effort Baseline 23

5 Risk Management and RMMM **24**
5.1 Risk Management Overview 24
5.2 Risk Mitigation, Monitoring, and Management (RMMM) Plan 24
5.2.1 Risk Mitigation Measures 24
5.2.2 Risk Monitoring Activities 24
5.2.3 Risk Response Planning 25
5.3 Project Risk Register and Response Plan 25
5.4 Risk Information Sheet Templates 26

6 Function Point Analysis and COCOMO Model Estimation **27**
6.1 Function Point Analysis (FPA) Method 27

6.1.1 Function Point Analysis Overview 27
6.1.2 Function Point Component Categories 27
6.1.3 Calculation of Unadjusted Function Points 28
6.1.4 Illustrative Function Point Count for Grambandhan 28
6.1.5 VAF and Adjusted Function Point Calculation 29
6.2 Constructive Cost Model (COCOMO) 30
6.2.1 COCOMO Model Overview 30
6.2.2 COCOMO Project Modes 30
6.2.3 Basic, Intermediate, and Detailed COCOMO 30
6.2.4 Basic COCOMO Equations and Parameters 30
6.2.5 Worked Basic COCOMO Example 31
6.2.6 Comparison of Effort Across Project Modes 32
6.2.7 Interpretation of COCOMO Results 32

7 Gantt Chart and Project Schedule 32
7.1 Development of the Gantt Chart 32
7.1.1 Estimation of Duration, Effort, and Overhead 33
7.1.2 Task Dependencies and Total Float 33
7.1.3 Proposed Schedule Baseline 33
7.1.4 Horizontal Gantt Chart 34

8 Change Control Management 35
8.1 What is Change Control 35
8.2 Situations Requiring Change Control 35
8.3 Change Management Models 36
8.3.1 Lewin's Three-Stage Change Model 36
8.3.2 ADKAR Model for Change Adoption 36
8.4 Reasons for Choosing the ADKAR Model 36
8.5 Steps in the Change Control Workflow 36

9 Software Process Improvement 37
9.1 What is Process Improvement 37
9.2 Six Sigma Methodology 37
9.2.1 Tools of Six Sigma 38
9.3 ISO 9000 Quality Management Principles 38

9.3.1 Eight Core Quality Management Principles 38

10 Capability Maturity Model Integration (CMMI) 39

10.1 What is CMMI 39

10.2 Objectives of the CMMI Framework 39

10.3 Three Main Areas Addressed by CMMI 39

10.4 CMMI Maturity-Level Structure 39

11 Project Management Tools and Selection 40

11.1 What are Project Management Tools 40

11.2 Comparison of Project Management Tools 40

11.3 Selected Project Management Tools for Grambandhan 40

12 Software Review Techniques 41

12.1 What is a Review 41

12.2 Formal Inspection 41

12.2.1 Roles in an Inspection Meeting 41

12.2.2 Stages of the Inspection Process 42

12.3 Desk Checking 42

12.4 Walkthroughs 42

12.5 Code Reviews 42

12.6 Collaborative Pair Programming 42

12.7 Participants in a Walkthrough 43

12.8 Code Review Methods 43

12.9 Features of Pair Programming 43

12.10Selected Review Techniques for Grambandhan 43

13 Conclusion and Future Work 43

13.1 Conclusion 43

13.2 Key Learnings 44

13.3 Future Scope 44

13.4 Final Remarks 44

1 Project Overview

Grambandhan is a proposed Smart Rural Investment Platform designed to connect rural producers with individuals who are interested in supporting agricultural and small rural business projects. The platform aims to provide a common digital space where project owners can present their plans and potential investors can review available opportunities.

In many rural areas, obtaining financial support for agricultural activities can be difficult. Farmers may need funds for seeds, fertilizers, equipment, and other essential resources, but finding suitable investors is not always straightforward. At the same time, people interested in rural investment may find it difficult to identify suitable projects and understand how those projects are progressing.

Grambandhan is intended to make this process more organized by bringing project information and related services into one platform. Project owners can present their proposals, potential investors can review the available details, and progress information can help participants follow project activities.

The project also considers supporting services such as an online marketplace, digital payment integration, risk-related suggestions, and fraud protection. Together, these capabilities are intended to improve communication between rural producers, investors, and potential buyers.

1.1 Problem Statement

Agricultural activities often require financial support before the work can begin. Farmers may need to purchase seeds, fertilizers, machinery, and other materials while also covering labour and transportation expenses. However, access to suitable funding can be limited, especially when project owners have difficulty reaching people outside their local networks.

Potential investors face a related problem. They may be interested in supporting rural projects but lack sufficient information to compare opportunities or understand how a project is progressing. When project details, funding requirements, expected expenses, and progress updates are not communicated clearly, making an informed decision becomes more difficult.

Communication can remain a challenge after a project starts. Project owners need a practical way to share updates, while investors may want to understand whether planned activities are moving forward. Inconsistent information can create misunderstandings and reduce confidence between the parties involved.

Grambandhan proposes a centralized platform where rural producers can present project information and potential investors can review the available details. By supporting project listings, progress updates, and organized communication, the platform aims to make rural investment more accessible and easier to follow.

The platform cannot remove every financial risk or guarantee the success of a project. Its

purpose is to provide a more structured way to share information and support decision-making.

1.2 Vision

The vision of Grambandhan is to make rural investment more accessible, transparent, and understandable for the people involved. The platform aims to help rural producers present their projects to a wider audience and give potential investors a convenient way to review available opportunities.

Another important goal is to improve communication between project owners and investors. Clear project information and regular progress updates can help participants understand the status of a project and identify issues that may need attention.

Grambandhan also seeks to create opportunities for rural producers to reach customers through digital marketplace facilities. By bringing investment-related information and supporting services into a common environment, the project aims to encourage stronger connections between rural businesses, investors, and buyers.

The long-term value of the platform will depend on the accuracy of the information provided, appropriate security measures, responsible participation, and whether the services meet the actual needs of users.

1.3 Scope and Features

The proposed scope of Grambandhan covers the main functions needed to connect rural project owners, investors, and potential buyers. The following features describe the intended scope of the platform.

- **Project Listing and Information:** Rural producers can present projects with descriptions, funding requirements, expected schedules, and other relevant details.
- **Investment and Profit-Sharing Workflow:** The platform concept includes facilities for presenting investment terms and recording participation. Expected returns, responsibilities, and possible losses should be communicated clearly.
- **Project Progress Tracking:** Project owners may share progress updates, photographs, and reports so that participants can follow project activities.
- **Online Agricultural Marketplace:** Rural producers can list agricultural products and other eligible goods for potential buyers.
- **Digital Payment Integration:** Integration with payment services such as bKash or Nagad may support convenient transactions. Actual payment processing depends on provider approval, API availability, and security requirements.

- **AI-Based Risk Suggestions:** AI-based suggestions are considered as a possible way to help users review project-related risks. Such suggestions require suitable validation and should not replace careful human assessment.
- **Fraud Protection and Verification:** Verification procedures, transaction records, and reporting mechanisms may help identify suspicious activity and support safer participation.
- **Insurance and Risk Support:** Insurance-related services may be considered for relevant agricultural risks, subject to suitable service arrangements and applicable requirements.
- **Ratings and Feedback:** Feedback facilities may allow users to share their experiences and identify areas for improvement.
- **Support for Rural Enterprises:** The platform may provide greater visibility for small rural businesses, including women-led enterprises and handicraft producers.

The scope may be refined as requirements become clearer. Features described as planned should only be reported as implemented after the team has verified that the corresponding functions have been developed and tested.

2 Methodology

A software development methodology provides a structured approach to planning, designing, implementing, testing, and maintaining a software system. It helps a team organize its work, assign responsibilities, manage changes, and evaluate progress against project goals.

Different methodologies suit different situations. Some work best when requirements are stable and can be defined in advance, while others allow more frequent feedback and changes during development. The choice depends on the project scope, team experience, available resources, and the level of uncertainty in the requirements.

Grambandhan includes several related areas, such as user management, project listings, investment workflows, progress tracking, marketplace functions, and possible external service integrations. The development approach should therefore allow the team to organize these areas into manageable tasks and review the results as work progresses.

The following sections discuss common software development methodologies and their possible relevance to Grambandhan.

2.1 What is Methodology

A software development methodology is a set of practices and guidelines that helps a team organize the process of building a software product. It provides a way to handle

requirements, design, implementation, testing, and delivery.

A suitable methodology can make project responsibilities clearer and help the team identify problems before they become difficult to manage. It also provides a common approach for communicating progress and deciding what work should be done next.

For Grambandhan, selecting an appropriate methodology is important because the platform contains several connected features. The team needs to consider the dependencies between these features, the feedback available from intended users, and the effort required for development and testing.

2.2 Waterfall Model

The Waterfall Model is a sequential approach in which software development proceeds through a series of stages. Each stage is generally completed before the team moves on to the next one. Requirements and design decisions are documented early, providing a reference for implementation and testing.

The typical stages include requirement analysis, system design, implementation, testing, deployment, and maintenance.

2.2.1 Waterfall Model Design

If the Waterfall Model were applied to Grambandhan, the team would begin by documenting the requirements of rural producers, investors, field agents, and administrators. The system design would then define the application structure, database, interfaces, access permissions, and major workflows.

After the design was approved, developers would implement the planned features. Testing would follow implementation, and the completed system would be prepared for deployment. Maintenance would address problems and changes after release.

This sequence can be useful when requirements are well understood and unlikely to change. However, features that depend on user feedback or external integrations may require additional flexibility.

2.2.2 Advantages of the Waterfall Model

- **Organized Development:** Clearly defined stages make the process easier to understand and follow.
- **Detailed Documentation:** Requirements and design decisions can be recorded before implementation begins.
- **Progress Measurement:** Completed stages provide checkpoints for reviewing project progress.

- **Stable Requirements:** The model can work well when the scope is clearly defined at the beginning.

2.2.3 Limitations of the Waterfall Model

- **Limited Flexibility:** Changes introduced late in development may require significant rework.
- **Delayed User Feedback:** Users may not be able to review a working version until later stages.
- **Late Problem Discovery:** Some design or usability issues may remain hidden until testing begins.
- **Dependence on Initial Requirements:** Incomplete requirements can lead to changes that affect previously completed work.

2.3 Scrum Development Methodology

Scrum is an Agile framework that organizes work into short, time-boxed periods called sprints. At the start of a sprint, the team selects tasks from a prioritized backlog. The team then works on those tasks and reviews the results before planning the next sprint.

The approach emphasizes regular communication, inspection of progress, and adaptation when new information becomes available.

2.3.1 Roles and Responsibilities in Scrum

Scrum defines responsibilities that help the team organize product development.

- **Product Owner:** Helps prioritize the product backlog and explains the value expected from planned features.
- **Scrum Master:** Supports the team in understanding Scrum practices and helps remove obstacles that affect progress.
- **Developers:** Work together to design, implement, test, and deliver the selected product features.

2.3.2 How the Scrum Workflow Operates

The Scrum workflow begins with a prioritized product backlog. During sprint planning, the team selects work that can reasonably be completed during the upcoming sprint.

Team members coordinate their work throughout the sprint and discuss obstacles that need attention. At the end of the sprint, the completed work is reviewed and feedback is

collected. A retrospective then gives the team an opportunity to discuss improvements to its working process.

For Grambandhan, this approach could help organize work on user registration, project listings, progress tracking, and marketplace features. Each sprint could focus on a manageable group of functions, depending on task dependencies and team capacity.

2.3.3 Advantages of Scrum Development

- **Regular Feedback:** Frequent reviews help the team learn from completed work and improve later increments.
- **Adaptability:** Priorities can be revised when requirements or project conditions change.
- **Visible Progress:** Small releases make it easier to demonstrate completed features.
- **Team Coordination:** Regular communication helps identify dependencies and resolve problems.

2.3.4 Challenges of Scrum Development

- **Regular Participation:** Planning, reviews, and coordination require consistent involvement from team members.
- **Backlog Management:** Poorly prioritized tasks can make it difficult to decide what should be developed first.
- **Changing Scope:** Frequent changes may affect sprint goals if priorities are not managed carefully.
- **Feedback Dependency:** Delayed stakeholder feedback can slow decisions about later work.

2.4 Prototype Methodology

The Prototype Model involves creating an early representation of a system so that users and developers can explore requirements before completing the full product. A prototype may be a screen mock-up, a clickable interface, or a limited working model.

The main purpose is to make the proposed system easier to discuss and evaluate. Feedback from users can help reveal missing requirements and confusing design choices before full implementation begins.

2.4.1 Designing a Prototype for Grambandhan

For Grambandhan, prototypes could be prepared for the project listing page, investor dashboard, progress tracking interface, and marketplace. These screens serve different users, so early review may help the team understand what information each user needs.

For example, a rural producer may need a simple way to describe a project, while an investor may need to review its funding target, expected schedule, and progress information. An early interface design can help identify these differences before the final functions are developed.

After collecting feedback, the team can revise the design and use the clarified requirements to guide implementation. A prototype should not be treated as a complete or production-ready system.

2.4.2 Advantages of the Prototype Model

- **Clearer Requirements:** Users can respond to a visible design instead of relying only on written descriptions.
- **Early Usability Review:** Confusing screens and missing information can be identified before full development.
- **Improved Communication:** A prototype provides a common reference for developers and stakeholders.
- **Design Exploration:** Different interface ideas can be evaluated before selecting a final direction.

2.4.3 Limitations of the Prototype Model

- **Repeated Revisions:** Frequent changes to the prototype can consume time and delay other work.
- **Unrealistic Expectations:** Users may assume that a prototype already has the reliability and security of the final product.
- **Limited Technical Detail:** A visual model may not reveal performance, database, or integration challenges.
- **Scope Expansion:** New ideas introduced during feedback can increase the amount of work beyond the original plan.

2.5 Incremental Model

The Incremental Model develops a system through a series of smaller releases. Each increment adds a defined group of functions to the existing product. This allows the

team to test and review individual parts rather than waiting until the entire system is complete.

The increments should be planned carefully because some features depend on others. For example, project progress tracking may depend on user accounts and project records being available first.

2.5.1 Planning Development Increments

A possible development sequence for Grambandhan would begin with the basic application structure and user management. The next increment could add project listings, followed by investment-related workflows and progress tracking.

Marketplace functions and other supporting services could then be developed according to their priority and technical dependencies. Each increment would be tested before it is integrated with the existing system.

The following sequence is a proposed plan rather than a claim about the exact order of completed work.

1. **Increment 1 – Basic Platform:** Prepare the application structure and navigation.
2. **Increment 2 – User Management:** Develop the planned registration, login, and role-based access functions.
3. **Increment 3 – Project Listings:** Add functions for creating and reviewing agricultural project information.
4. **Increment 4 – Investment and Tracking:** Develop the planned investment workflow and project progress updates.
5. **Increment 5 – Supporting Services:** Add prioritized marketplace, payment, feedback, or related functions.
6. **Increment 6 – Integration and Testing:** Test the combined system, correct defects, and prepare the intended release.

2.5.2 Advantages of the Incremental Model

- **Manageable Work Packages:** Large requirements are divided into smaller groups that are easier to plan.
- **Earlier Evaluation:** Individual features can be tested before the entire system is finished.
- **Flexible Priorities:** Feedback from earlier increments can guide later development decisions.

- **Visible Progress:** The team can monitor individual features and identify unfinished dependencies.

2.5.3 Limitations of the Incremental Model

- **Integration Challenges:** Separately developed components may not work together without careful coordination.
- **Planning Requirements:** Dependencies between features must be identified before the work is divided into increments.
- **Architecture Changes:** Changes to the system structure may affect previously developed components.
- **Repeated Testing:** Regression testing may be needed after new features are integrated.

2.6 Extreme Programming (XP) Methodology

Extreme Programming (XP) is an Agile methodology that emphasizes software quality, frequent feedback, simple design, and close collaboration. It encourages developers to test their work regularly and improve the code as requirements become clearer.

Common XP practices include small releases, automated testing, refactoring, pair programming, and continuous integration. A team may use selected practices where they fit its working environment.

2.6.1 Applying XP Practices to Grambandhan

For Grambandhan, selected XP practices could support functions such as authentication, project data validation, and investment records. Automated tests could check how the system handles valid and invalid input, while code reviews could help identify problems before changes are integrated.

Frequent integration would also help reveal conflicts between separately developed features. Refactoring could be used when existing code becomes difficult to understand or maintain.

These are possible applications of XP practices. The report should only state that a specific practice was adopted if it reflects the team's actual work.

2.6.2 Advantages of Extreme Programming

- **Frequent Testing:** Regular tests can help detect defects during development.
- **Continuous Feedback:** Small releases make it easier to review changes and improve the product.

- **Maintainable Code:** Refactoring can improve code structure without intentionally changing its external behaviour.
- **Shared Knowledge:** Pair programming and code reviews can help developers learn from one another.

2.6.3 Challenges of Extreme Programming

- **Team Discipline:** Testing and integration practices need to be followed consistently.
- **Coordination Effort:** Close collaboration can require additional time and scheduling.
- **Test Maintenance:** Automated tests need to be updated as the system and its requirements change.
- **Documentation Needs:** Important design decisions and requirements still need to be recorded when necessary.

2.7 Our Project Methodology

For Grambandhan, incremental development is a reasonable proposed approach because the platform contains several related features that can be organized into smaller groups. The team can focus on the basic platform and user management first, then develop project listings and other functions according to their priorities and dependencies.

Prototyping can help clarify interface requirements before the complete features are implemented. Regular testing and code review can also help the team identify defects as different components are developed and integrated.

Scrum-style planning and review activities may help organize tasks and discuss progress, while selected XP practices may support testing and code quality. However, these practices should not be described as formally adopted unless the team actually used them during development.

Therefore, an incremental approach supported by appropriate planning, prototyping, and testing is proposed for Grambandhan. The final description should reflect the methodology the team actually followed and the work that has been completed.

2.8 Methodology Phases

The development process for Grambandhan can be organized into the following phases. Together, these phases provide a structured way to connect the project requirements with implementation, testing, and release preparation.

1. **Requirement Analysis:**

Identify the problem, intended users, system functions, project constraints, and requirements of rural producers, investors, field agents, and administrators.

2. **Planning and Prioritization:**

   Divide the requirements into manageable tasks, identify dependencies, assign responsibilities, and decide which features should be addressed first.

3. **System Design and Prototyping:**

   Prepare the system structure, data model, interface designs, and important workflows. Prototypes may be used to review the proposed interfaces and clarify requirements.

4. **Implementation:**

   Develop the selected features in manageable groups. Each work package should have clear requirements and acceptance criteria.

5. **Integration and Testing:**

   Test individual functions and their interaction. Relevant checks may include input validation, access permissions, error handling, usability, and integrations included in the implementation.

6. **Review and Improvement:**

   Examine completed work, record identified problems, collect available feedback, and revise remaining tasks when necessary.

7. **Deployment and Maintenance:**

   Prepare the required configuration, documentation, and user guidance. After release, address reported defects and consider justified improvements.

Organizing development in these phases can help the team monitor progress and maintain a clearer connection between requirements, implementation, and testing. The detailed schedule and final deliverables should be confirmed using the team's actual project plan.

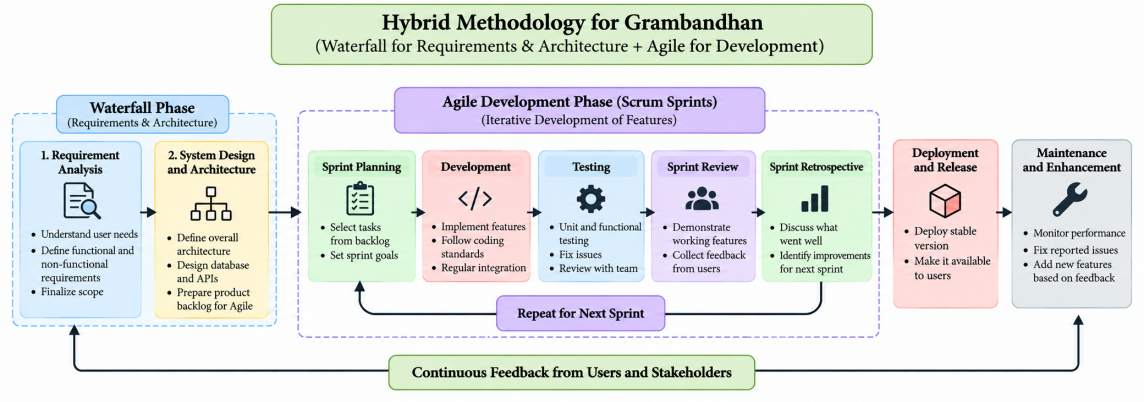


Figure 1: Proposed Hybrid Development Model for Grambandhan

3 Project Work Decomposition

3.1 Purpose and Structure of the WBS

A Work Breakdown Structure (WBS) divides the total project scope into smaller and more manageable deliverables. It helps the team understand what needs to be produced, organize responsibilities, estimate effort, and monitor progress.

For Grambandhan, the WBS connects the overall project goal with specific deliverables such as requirements analysis, system design, user management, project listings, investment-related functions, marketplace services, testing, and deployment preparation.

The WBS also helps the project team coordinate its work. By identifying the major deliverables and their subtasks, the team can better understand which work needs to be completed and how different parts of the platform depend on one another.

3.2 Guidelines for Organizing Project Deliverables

A useful WBS should cover the agreed project scope, describe clear deliverables, and divide the work into manageable packages. Each work package should have a clear purpose so that the team can estimate the required effort and identify responsibility.

For Grambandhan, the WBS includes project coordination, requirements, system design, core platform development, project and investment features, marketplace services, testing, and release preparation.

The following guidelines help maintain a clear structure and reduce confusion during development.

3.2.1 Complete Scope Coverage

The WBS should include all the work required to complete the agreed project scope. This includes development activities as well as supporting work such as planning, requirements analysis, testing, documentation, and release preparation.

For Grambandhan, the WBS should not include only programming tasks. Work related to security, integration, quality assurance, and documentation must also be considered when these activities are part of the approved project scope.

3.2.2 Defining Deliverables Clearly

WBS elements should describe the deliverables that the team expects to produce rather than using vague activity descriptions.

For example, “Project Listing Module” identifies a deliverable that can be reviewed against specific requirements. In contrast, “Work on Project Listing” does not clearly

describe what must be completed.

For Grambandhan, using deliverable-based task names helps the team understand the expected result and makes progress easier to review.

3.2.3 Establishing the Main Deliverable Groups

The second level of the WBS identifies the major deliverable groups within the project. It provides a useful overview without including every small implementation detail.

For Grambandhan, the main groups include project management, requirements analysis, system design, core platform development, project and investment features, marketplace services, testing, and release preparation.

These groups help the team organize its work and identify the major areas that require coordination.

3.2.4 Information Required for Each Work Package

Each WBS element should have four important details:

1. **WBS Code:** A unique identifier for the work package.
2. **Deliverable Description:** A clear description of the expected output.
3. **Responsible Member:** The team member assigned to coordinate or lead the work package.
4. **Acceptance Criteria:** The conditions used to check whether the deliverable meets the agreed requirements.

For example, the project listing module can have a unique WBS code, a description of its functions, an assigned team member, and acceptance criteria covering the required project information and validation rules.

These details help the team understand what needs to be delivered and how completion will be verified.

3.2.5 Avoiding Overlap Between Work Packages

WBS elements should not overlap unnecessarily. Overlapping responsibilities can lead to duplicated work, unclear ownership, and inconsistent estimates.

For Grambandhan, authentication logic can be assigned to the core platform work package, while the usability of the login interface can be considered during interface design and testing. Although these tasks are related, their responsibilities should be defined clearly.

Clear boundaries help the team coordinate its work without unnecessary duplication.

3.2.6 Breaking Down Large Tasks

A work package that is too large may be difficult to estimate, assign, or monitor. One practical guideline is to review work packages that require more than approximately one person-week of effort and divide them when doing so improves project control.

For example, the project and investment module could be divided into project creation, funding information, investment records, and progress tracking.

This is a planning guideline rather than a strict requirement. The level of decomposition should depend on the complexity of the task and the team's needs.

3.2.7 Reviewing Work Package Size

A WBS should avoid work packages that are too large to estimate or monitor effectively. A common planning guideline is to review the smallest work packages when they represent a substantial share of the total project effort.

For Grambandhan, this guideline can be used when reviewing the size of individual tasks. Larger tasks may be divided into smaller deliverables when doing so improves estimation and responsibility assignment.

The appropriate size of a work package depends on the project scope, the level of detail required, and the team's working arrangements. The final structure should remain practical rather than breaking every task into unnecessary small activities.

3.3 Grambandhan Deliverable Breakdown

The following table presents a proposed WBS for Grambandhan. It identifies the major deliverables, their subtasks, and preliminary durations.

The durations are initial planning estimates. Some tasks may run in parallel, while others depend on the completion of earlier work. These estimates should be reviewed against the team's actual schedule before being treated as final.

Table 1: Proposed WBS for Grambandhan

WBS Code	**Task Name**	**Subtasks / Deliverables**	**Duration**
1.0	**Project Coordination**	Stakeholder needs and project scope	2 weeks
1.1		Functional requirements and constraints	1 week
1.2		Existing-platform review	1 week
1.3		Requirement review and approval	1 week

Continued on next page

WBS Code	Task Name	Subtasks / Deliverables	Duration
2.0	**Requirements Analysis**	Application architecture and interface design	3 weeks
2.1		UI/UX mock-ups and prototypes	1 week
2.2		Backend structure and API design	1 week
2.3		Database schema and data relationships	1 week
2.4		UML diagrams and process-flow design	2 weeks
3.0	**System Architecture and Interface Design**	Core user and access-control functions	3 weeks
3.1		Account creation and secure sign-in	2 weeks
3.2		User roles and permission management	2 weeks
3.3		Profile review and identity confirmation	2 weeks
3.4		Administrator tools and access control	1 week
4.0	**User and Role Management**	Project listing and investment workflow	4 weeks
4.1		Project creation and listing updates	1 week
4.2		Funding targets and investment terms	1 week
4.3		Investment records and status tracking	2 weeks
4.4		Profit-sharing terms and project records	2 weeks
5.0	**Project and Investment Workflow**	Progress tracking and reporting	4 weeks
5.1		Dated project progress updates	2 weeks
5.2		Project status and report display	2 weeks
5.3		Progress media and report uploads	2 weeks
5.4		Investor progress dashboard	2 weeks
6.0	**Marketplace and Supporting Services**	Marketplace listing and order workflow	4 weeks
6.1		Product listing and search	1 week
6.2		Cart, order, and buyer-seller interaction	2 weeks
6.3		Marketplace order records	2 weeks
6.4		Risk suggestions and fraud controls	2 weeks
6.5		Feedback and suspicious-activity reporting	1 week

Continued on next page

WBS Code	Task Name	Subtasks / Deliverables	Duration
7.0	**Verification and Quality Assurance**	Testing and system validation	3 weeks
7.1		Functional and integration testing	1 week
7.2		Security and transaction-status checks	1 week
7.3		Defect correction and regression checks	1 week
7.4		Usability and acceptance review	1 week
8.0	**Release and Documentation**	Deployment preparation and release records	2 weeks
8.1		Deployment configuration and verification	1 week
8.2		User guide and technical documentation	1 week
8.3		Final review and release preparation	1 week

The WBS provides a structured view of the major deliverables and their subtasks. The durations are initial planning estimates and should be reviewed against the team's actual schedule. Some subtasks may overlap or depend on earlier work, so their durations should not be added together blindly.

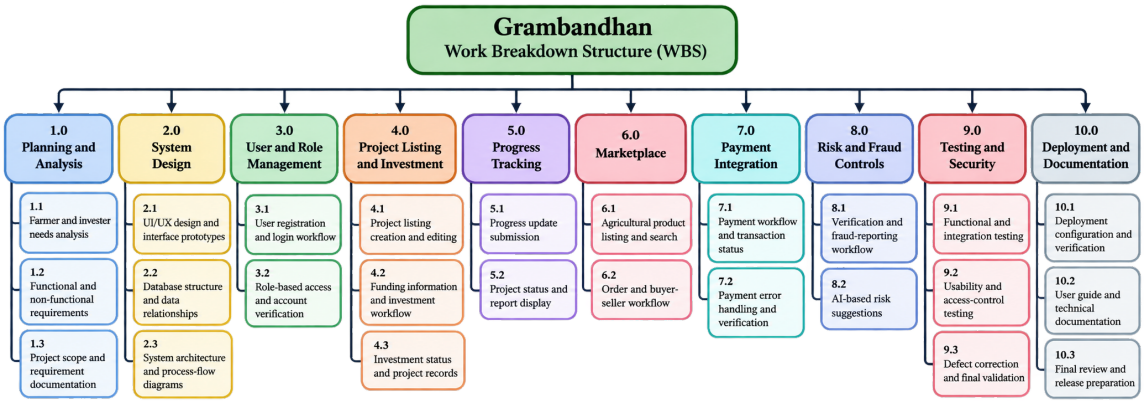


Figure 2: Hierarchical Work Breakdown Structure (WBS) of Grambandhan

4 Wideband Delphi Estimation

4.1 Wideband Delphi Estimation Technique

Wideband Delphi is a group-based estimation technique in which team members independently estimate the effort required for defined project tasks. The estimates are compared, differences are discussed, and task definitions may be refined before another estimation

round. This approach helps make assumptions visible and avoids depending entirely on a single person's judgment.

For Grambandhan, the technique is relevant because the platform combines user verification, project listing and investment, progress tracking, marketplace services, digital payments, and risk-related features. These modules have different levels of complexity and integration needs, so estimating them separately provides a more useful basis for planning.

4.2 Steps in the Wideband Delphi Process

4.2.1 Selection of the Estimation Team

The estimation group should include members who understand the project requirements and planned deliverables. The Grambandhan team consists of Sourav, Jony, Supty, Partha, and Upol. A moderator should coordinate the sessions, collect estimates, record assumptions, and make sure that every participant can explain the reasoning behind an estimate.

4.2.2 Estimation Kick-off Meeting

At the kick-off meeting, the moderator reviews the project scope, WBS, task boundaries, and estimation unit. The team should agree on whether estimates represent person-hours, person-days, or elapsed duration. For the current baseline, the available task values are recorded in hours; they should be interpreted as effort only if that is how the original values were prepared.

4.2.3 Independent Task Estimation

Each participant reviews the work packages and independently estimates the effort required. The estimate should consider analysis, design, implementation, integration, testing, and documentation where applicable. Participants should also note assumptions such as the availability of payment-provider documentation, the stability of requirements, and the time needed for project verification.

The main areas to estimate for Grambandhan are:

- Account registration, authentication, and role verification.
- Project listing, funding targets, investment records, and profit-sharing terms.
- Progress updates, media uploads, and investor monitoring.
- Agricultural product listings, search, cart, and order handling.
- Payment status management and transaction reconciliation.

- Risk suggestions, project verification, fraud reporting, and audit records.
- Integration, security testing, deployment, and documentation.

4.2.4 Comparison and Discussion of Estimates

The moderator compares the independent estimates and highlights tasks with noticeable differences. The team discusses the causes, such as uncertain requirements, hidden integration work, validation effort, or differing assumptions about implementation. After clarification, participants may revise their estimates. Both the original and revised values should be retained so the reasoning remains traceable.

4.2.5 Consolidation of Work Packages

The moderator consolidates the task list, removes duplicated work, and aligns task descriptions with the WBS. Each estimate should correspond to a defined deliverable, and the combined list should cover the agreed project scope without overlapping work packages.

4.2.6 Review of Estimation Results

The team checks whether the estimates include necessary integration, testing, security, and documentation work. If estimates remain far apart, the relevant tasks should be clarified and another round conducted. Values should only be described as a Delphi consensus after the team has completed the independent estimation and review process.

4.2.7 Initial Task Effort Summary

The table below preserves the available Grambandhan task estimates as an initial planning baseline. It does not claim that the values were independently collected from every team member.

Table 2: Initial Effort Baseline for Grambandhan

Task ID	Task Description	Hours
WD-01	User registration and role-based verification	45
WD-02	Project listing and profit-sharing system	60
WD-03	Real-time project tracking using images, videos, and reports	55
WD-04	Online marketplace and cart/order workflow	65
WD-05	AI-based project risk suggestion feature	70
WD-06	Digital payment integration and escrow-related workflow	50
WD-07	Fraud protection and project verification	40
	Total Listed Effort	**385**

The total listed effort is:

$$E_{\text{total}} = 45 + 60 + 55 + 65 + 70 + 50 + 40 = 385 \text{ hours}$$

The AI-based risk suggestion feature has the highest listed effort, while fraud protection and project verification have the lowest. These values are an initial baseline and should be reviewed against the detailed requirements.

4.3 Individual Three-Point Estimation Form

A completed Wideband Delphi exercise requires each participant's own estimates. The following form is intended to record those values without inventing personal estimates. Complete one copy for each participant before calculating a consolidated result.

Table 3: Individual Estimation Form (Person-Days)

Task ID	**Work Package**	**Low**	**Most Likely**	**High**
WD-01	User registration and verification	2	4	6
WD-02	Project listing and investment	3	5	8
WD-03	Progress tracking	3	6	9
WD-04	Marketplace and orders	4	7	11
WD-05	AI-based risk suggestions	5	8	13
WD-06	Payment workflow	3	5	8
WD-07	Fraud protection and verification	2	4	6

For three-point estimation, the PERT expected effort for each task can be calculated using:

$$E_{\text{PERT}} = \frac{L + 4M + H}{6}$$

The corresponding standard deviation is:

$$\sigma = \frac{H - L}{6}$$

where L is the low estimate, M is the most-likely estimate, and H is the high estimate. These equations should be applied to the actual values submitted by the participants. The 385-hour total above is a separate initial task baseline and must not be presented as the result of this formula unless the required three-point estimates are available.

4.4 Analysis of the Initial Effort Baseline

The 385-hour baseline helps identify which work packages may need additional planning. Project listing, marketplace development, and AI-based risk suggestions have comparatively high estimates, while payment and verification tasks also need careful integration

and testing. The baseline should not be treated as a confirmed Delphi consensus until the team has completed individual estimates, discussed differences, and recorded agreed values.

5 Risk Management and RMMM

5.1 Risk Management Overview

Risk management is the ongoing process of identifying uncertain events, evaluating their likelihood and impact, planning responses, and monitoring changes throughout a project. It helps the team reduce avoidable problems and prepare for events that could affect scope, schedule, quality, budget, or stakeholder confidence.

Grambandhan connects rural project owners with investors and buyers. As a result, risks may arise from digital literacy, unstable connectivity, inaccurate project information, payment-provider dependency, fraud, agricultural uncertainty, and the handling of personal or financial information.

5.2 Risk Mitigation, Monitoring, and Management (RMMM) Plan

RMMM combines three related activities. Risk mitigation reduces the probability or impact of a risk before it occurs. Risk monitoring checks indicators and identifies changes in exposure. Risk management and planning define the actions to take if a risk becomes active.

5.2.1 Risk Mitigation Measures

Mitigation measures for Grambandhan include simple user workflows, mobile-friendly screens, project and user verification, careful transaction-status handling, role-based access control, and a clear reporting mechanism. Payment completion should only be confirmed after the payment service returns a valid confirmation. AI-generated risk suggestions should be communicated as guidance rather than guaranteed predictions.

5.2.2 Risk Monitoring Activities

The team should monitor incomplete registration, failed verification, payment errors, inconsistent transaction statuses, reported project concerns, upload failures, and defects found during testing. These indicators can be reviewed during progress meetings and before demonstrations. Repeated problems should be documented and assessed for their effect on project delivery and user trust.

5.2.3 Risk Response Planning

Each significant risk should have a likelihood rating, impact rating, preventive action, warning trigger, and contingency response. High-impact risks deserve attention even when their probability is considered low. When a risk occurs, the team should record the incident, apply the response, communicate relevant changes, and verify whether the problem has been resolved.

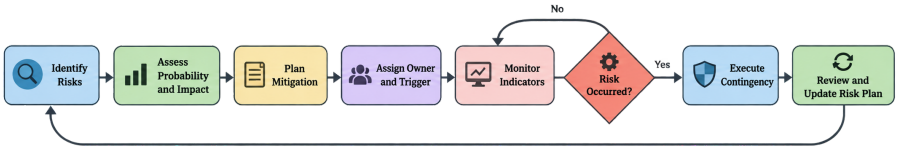


Figure 3: Risk Mitigation, Monitoring, and Management Cycle for Grambandhan

5.3 Project Risk Register and Response Plan

The following risk plan focuses on operational and technical concerns relevant to Grambandhan.

Table 4: Risk Register and Response Plan for Grambandhan

ID	Risk	Likelihood	Impact	Response and Monitoring
R1	Digital literacy limitations	Medium	High	Use plain language, clear navigation, mobile-friendly screens, and onboarding guidance. Monitor incomplete forms and usability feedback.
R2	Weak or intermittent connectivity	High	Medium	Keep pages lightweight, handle interrupted submissions safely, show clear status messages, and test under constrained network conditions.
R3	Misleading project information	Medium	High	Verify essential project details, maintain update records, and provide a reporting and review workflow for disputed listings.

ID	Risk	Likelihood	Impact	Response and Monitoring
R4	Payment-provider failure or status mismatch	Medium	High	Validate payment responses, retain transaction references, reconcile records, and avoid marking uncertain transactions as successful.
R5	Agricultural loss due to weather, pests, or other factors	High	High	Communicate uncertainty, present risk guidance responsibly, and avoid implying guaranteed investment returns.
R6	Fraudulent accounts or investment claims	Medium	High	Apply account and project verification, retain audit records, and investigate suspicious reports before further action.
R7	Exposure of user or financial information	Medium	Very High	Enforce server-side role permissions, protect sensitive data, review access logs, and define an incident-response procedure.
R8	Scope expansion and schedule slippage	Medium	Medium	Prioritize essential deliverables, track dependencies, review estimates when requirements change, and defer optional features if necessary.
R9	Integration defects between modules	Medium	Medium	Define interface expectations early, test integrations incrementally, and resolve critical defects before release.

The likelihood and impact ratings are preliminary qualitative assessments. They should be reviewed as the requirements become clearer and as testing provides additional evidence.

5.4 Risk Information Sheet Templates

A risk information sheet documents one risk in greater detail, including its context, mitigation actions, monitoring indicators, contingency response, and current status.

Table 5: Risk Information Sheet: Connectivity Dependency

Field	**Details**
Risk ID	GB-R2
Probability / Impact	High / Medium
Description	Users may be unable to complete an operation when the network is weak or interrupted.
Context	Some intended users may access the platform through mobile networks with variable quality.
Mitigation / Monitoring	Optimize page size, show clear progress and status, and monitor failed or repeated submissions.
Contingency / Trigger	If repeated failures occur, provide a safe retry path and investigate the affected workflow before the next release.
Status	To be updated during team risk reviews.

Table 6: Risk Information Sheet: Payment Status Inconsistency

Field	**Details**
Risk ID	GB-R4
Probability / Impact	Medium / High
Description	The payment service and platform may show different states for a transaction.
Context	Transaction processing depends on an external provider and correct handling of callbacks or responses.
Mitigation / Monitoring	Store transaction references, validate provider responses, test failure scenarios, and reconcile records.
Contingency / Trigger	If the status is pending or conflicting, do not mark the payment successful until verification is complete.
Status	To be updated during team risk reviews.

6 Function Point Analysis and COCOMO Model Estimation

6.1 Function Point Analysis (FPA) Method

6.1.1 Function Point Analysis Overview

Function Point Analysis (FPA) measures the functional size of a software system from the user's perspective. It does not directly depend on the programming language. Function points are estimated by identifying and classifying the functionality provided by the system.

6.1.2 Function Point Component Categories

FPA uses five function types:

- **External Input (EI):** Data entering the system, such as registration, project submission, and product listing.
- **External Output (EO):** Processed information leaving the system, such as reports and investment summaries.
- **External Inquiry (EQ):** Requests that retrieve information with limited processing, such as project search and status lookup.
- **Internal Logical File (ILF):** Logical data groups maintained by Grambandhan, such as users, projects, investments, and orders.
- **External Interface File (EIF):** Data maintained by an external system but referenced by Grambandhan, where applicable.

6.1.3 Calculation of Unadjusted Function Points

Each function is assigned a complexity level (Low, Average, or High) and multiplied by its standard weight. The weighted values are added to obtain the Unadjusted Function Point total.

$$UFP = \sum_{i=1}^{5} \sum_{j=1}^{3} N_{ij} W_{ij}$$

where N_{ij} is the number of functions of type i at complexity level j, and W_{ij} is the corresponding standard weight.

Table 7: Standard Function Point Weights

Function Type	**Low**	**Average**	**High**
External Input (EI)	3	4	6
External Output (EO)	4	5	7
External Inquiry (EQ)	3	4	6
Internal Logical File (ILF)	7	10	15
External Interface File (EIF)	5	7	10

6.1.4 Illustrative Function Point Count for Grambandhan

The following table uses a preliminary function inventory for illustration. The final counts and complexity ratings must be checked against the approved requirements.

Using the example counts and weights shown above:

$$UFP = (4 \times 4) + (3 \times 5) + (3 \times 3) + (4 \times 10) + (1 \times 5)$$

Table 8: Illustrative Function Point Count for Grambandhan

Type	Example	Count	Weight Used	Subtotal
EI	Registration, project submission, investment request, product listing	4	4	16
EO	Project reports, investment summaries, payment status messages	3	5	15
EQ	Project search, project details, order-status lookup	3	3	9
ILF	User records, projects, investments, marketplace orders	4	10	40
EIF	External payment-provider/reference information, if within the defined boundary	1	5	5
			Illustrative UFP	**85**

$$UFP = 16 + 15 + 9 + 40 + 5 = 85$$

This value is illustrative only. It is not a verified final function-point count because the system boundary, full function inventory, and complexity classifications have not yet been validated.

6.1.5 VAF and Adjusted Function Point Calculation

In the traditional adjusted function-point method, the Value Adjustment Factor is calculated from ratings for 14 General System Characteristics (GSCs):

$$VAF = 0.65 + 0.01 \sum_{i=1}^{14} F_i$$

The Adjusted Function Point value is:

$$AFP = UFP \times VAF$$

where F_i is the rating assigned to the i-th General System Characteristic. Since verified GSC ratings have not been provided for Grambandhan, an adjusted FP value is not claimed here. The illustrative UFP of 85 should be replaced after the function inventory and complexity ratings are reviewed.

6.2 Constructive Cost Model (COCOMO)

6.2.1 COCOMO Model Overview

The Constructive Cost Model (COCOMO) estimates software development effort and schedule from an estimate of software size, traditionally measured in thousands of delivered source lines of code (KLOC). The result depends on the chosen COCOMO model, project mode, size estimate, and assumptions.

6.2.2 COCOMO Project Modes

- **Organic:** Relatively small projects developed by an experienced team in a familiar environment with few rigid constraints.
- **Semi-detached:** Projects of intermediate complexity with a mixed-experience team and moderate constraints.
- **Embedded:** Projects with tight hardware, operational, regulatory, or performance constraints.

The project mode for Grambandhan cannot be determined with certainty from the available information. The organic mode is used below as an illustrative baseline. The payment, security, and external-service requirements may increase the effective complexity and should be considered during final estimation.

6.2.3 Basic, Intermediate, and Detailed COCOMO

- **Basic COCOMO:** Uses KLOC and project mode to estimate effort and development time.
- **Intermediate COCOMO:** Adds an Effort Adjustment Factor based on cost drivers.
- **Detailed COCOMO:** Applies cost-driver effects and distributes effort across lifecycle phases.

6.2.4 Basic COCOMO Equations and Parameters

Basic COCOMO uses the following equations:

$$E = a(KLOC)^b$$

$$T_{DEV} = c(E)^d$$

$$P = \frac{E}{T_{DEV}}$$

where:

- E = estimated effort in person-months;
- $KLOC$ = thousands of delivered source lines of code;
- T_{DEV} = estimated development time in months;
- P = average nominal team size in persons;
- a, b, c, d = coefficients determined by the project mode.

Table 9: Basic COCOMO Coefficients

Project Mode	***a***	***b***	***c***	***d***
Organic	2.4	1.05	2.5	0.38
Semi-detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

6.2.5 Worked Basic COCOMO Example

The actual KLOC of Grambandhan has not been measured. To demonstrate the formula without claiming an unsupported code size, this calculation assumes $KLOC = 10$ and the organic mode.

Step 1: Calculate effort

$$E = a(KLOC)^b$$

$$E = 2.4(10)^{1.05}$$

$$E \approx 26.9 \text{ person-months}$$

Step 2: Calculate development time

$$T_{DEV} = c(E)^d$$

$$T_{DEV} = 2.5(26.9)^{0.38}$$

$$T_{DEV} \approx 8.7 \text{ months}$$

Step 3: Calculate average nominal team size

$$P = \frac{E}{T_{DEV}} = \frac{26.9}{8.7}$$

$$P \approx 3.1 \text{ persons}$$

6.2.6 Comparison of Effort Across Project Modes

The following comparison uses the same illustrative $KLOC = 10$ input for all three modes. It demonstrates how the selected coefficients affect the result; it does not assert that all three modes are equally suitable for Grambandhan.

Table 10: Illustrative Basic COCOMO Results for Different Modes

Project Mode	**Effort (PM)**	**Development Time (Months)**	**Average Team Size**
Organic	$2.4(10)^{1.05} \approx 26.9$	$2.5(26.9)^{0.38} \approx 8.7$	$26.9/8.7 \approx 3.1$
Semi-detached	$3.0(10)^{1.12} \approx 39.6$	$2.5(39.6)^{0.35} \approx 9.0$	$39.6/9.0 \approx 4.4$
Embedded	$3.6(10)^{1.20} \approx 57.1$	$2.5(57.1)^{0.32} \approx 9.5$	$57.1/9.5 \approx 6.0$

These figures are illustrative calculations only. A final COCOMO estimate requires an agreed KLOC estimate and a justified project mode. Intermediate or Detailed COCOMO also requires appropriate cost-driver ratings.

6.2.7 Interpretation of COCOMO Results

The example shows that estimated effort rises as the assumed project mode becomes more constrained. For Grambandhan, the team should first estimate the likely code size from the approved design and implementation scope, then choose and justify the project mode. The COCOMO estimate should be compared with the task-based Wideband Delphi baseline; differences should be explained rather than hidden.

7 Gantt Chart and Project Schedule

7.1 Development of the Gantt Chart

A Gantt chart presents project activities on a timeline. It helps the team see the planned duration of tasks, the order in which work is expected to happen, and the points where one deliverable depends on another.

7.1.1 Estimation of Duration, Effort, and Overhead

Duration describes the calendar time assigned to a task, while effort describes the amount of work required. They are not interchangeable: a task requiring several person-days may take longer if the people or resources needed are not available continuously. Review, coordination, integration, and correction time should also be considered.

7.1.2 Task Dependencies and Total Float

Grambandhan has dependencies between requirements, design, module implementation, integration, testing, and release preparation. For example, role-based access needs an agreed user model, and payment integration should be tested after the transaction workflow and relevant data structures are defined.

For a network schedule, the earliest and latest activity times can be used to calculate total float:

$$\text{Total Float} = LS - ES = LF - EF$$

where ES and EF are the earliest start and finish, and LS and LF are the latest start and finish. Activities with zero total float lie on a critical path under the selected schedule assumptions. The actual critical path cannot be identified until task dependencies and durations have been confirmed.

7.1.3 Proposed Schedule Baseline

The following schedule is an indicative baseline based on the project work packages. Some activities may overlap, so the durations should not simply be added to obtain the total calendar duration.

Table 11: Proposed Grambandhan Schedule Baseline

ID	Work Package	Indicative Duration	Dependency / Milestone
G1	Planning, stakeholders, and requirements	2 weeks	Project scope agreed
G2	System architecture, data model, and UI/UX design	3 weeks	G1
G3	User registration and role management	2 weeks	G2
G4	Project listing and investment workflow	4 weeks	G2–G3

ID	Work Package	Indicative Duration	Dependency / Milestone
G5	Progress tracking and reporting	3 weeks	G3–G4
G6	Marketplace and order processing	3 weeks	G3–G4
G7	Payment integration and reconciliation	3 weeks	G4 and transaction workflow
G8	Risk controls, verification, and fraud reporting	3 weeks	G3–G7 as applicable
G9	Integration, security, usability, and acceptance testing	3 weeks	Core modules available
G10	Deployment, documentation, and release preparation	2 weeks	G9

The schedule is a planning proposal rather than a verified historical timeline. A final Gantt chart should include actual start dates, end dates, resource availability, and confirmed dependencies.

7.1.4 Horizontal Gantt Chart

Figure 4 shows an indicative 16-week schedule for the main Grambandhan work packages. Each task is placed on its own row so that the timing and overlap can be read clearly. The schedule is provisional and should be updated after the team confirms actual dates, dependencies, and resource availability.

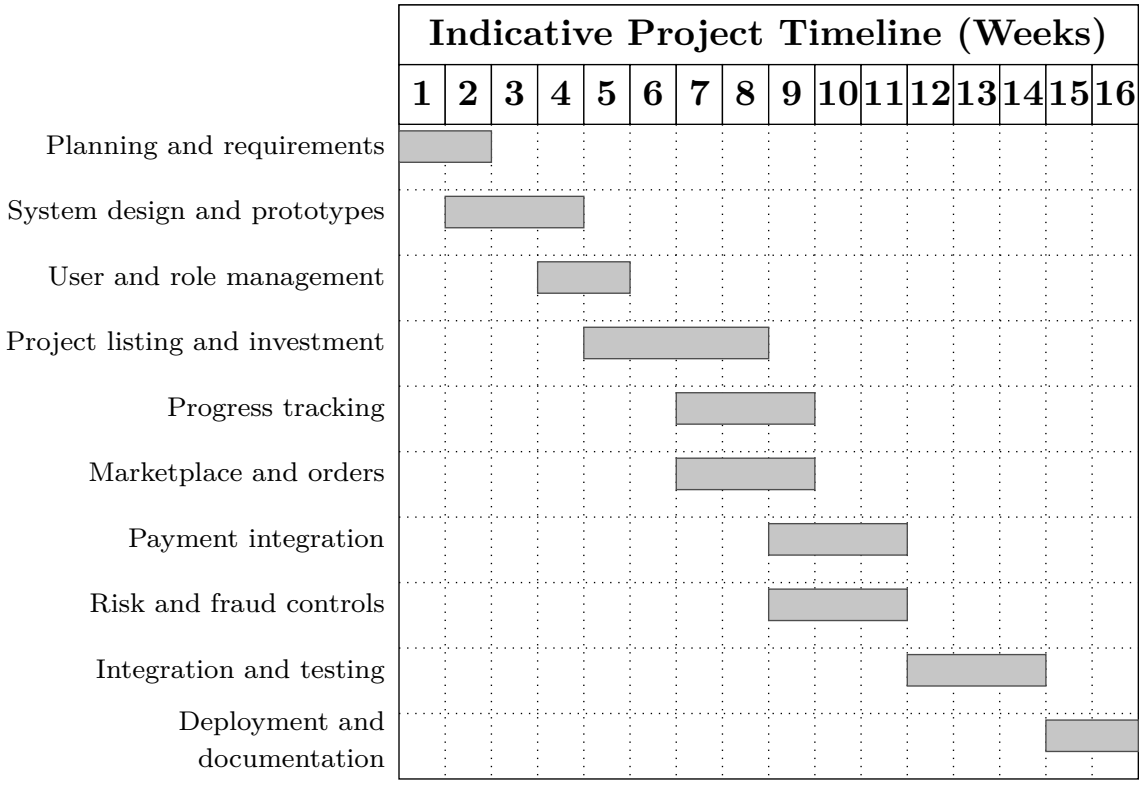


Figure 4: Horizontal Gantt Chart for the Proposed Grambandhan Schedule

The timeline intentionally allows design, implementation, and supporting functions to overlap where practical. Testing is scheduled after the main feature groups are available, followed by deployment preparation. This is a planning illustration rather than a confirmed critical-path schedule.

8 Change Control Management

8.1 What is Change Control

Change control is a structured process for reviewing proposed changes before they are added to an approved project baseline. It helps the team understand how a change may affect scope, effort, schedule, quality, security, and related deliverables.

For Grambandhan, change requests may arise when stakeholders request new roles, additional payment methods, changes to profit-sharing rules, extra marketplace features, or modifications to project-verification and risk-suggestion workflows.

8.2 Situations Requiring Change Control

A formal change request should be recorded when a proposed change affects an approved requirement, WBS item, acceptance criterion, schedule, integration, or security control.

Small editorial corrections that do not alter meaning can be handled by the document owner, but changes to functionality or commitments should be reviewed before implementation.

8.3 Change Management Models

8.3.1 Lewin's Three-Stage Change Model

Lewin's model describes change in three stages. **Unfreezing** prepares the team by explaining why the current approach needs to change. **Changing** involves evaluating alternatives and implementing the selected approach. **Refreezing** incorporates the accepted change into requirements, WBS items, tests, documentation, and team procedures.

For example, if testing shows that a project listing does not provide enough information for verification, the team first agrees on the problem, then designs and implements the required fields and review workflow, and finally updates the acceptance criteria and documentation.

8.3.2 ADKAR Model for Change Adoption

ADKAR describes five outcomes that support adoption: Awareness, Desire, Knowledge, Ability, and Reinforcement. In Grambandhan, the team should understand why a change is necessary, agree on its value, learn the updated workflow, demonstrate that it works, and reinforce it through tests, reviews, and documentation.

8.4 Reasons for Choosing the ADKAR Model

ADKAR is useful for a small project team because it connects technical changes with the people who must implement and use them. It can help distinguish whether a problem comes from a lack of understanding, motivation, knowledge, practical ability, or follow-through. It is especially relevant when changing workflows intended for farmers, investors, buyers, or administrators.

8.5 Steps in the Change Control Workflow

The following process script defines a repeatable way to handle proposed changes in Grambandhan.

Table 12: Change Control Process for Grambandhan

Element	Description
Purpose	Evaluate the effect of a proposed change before deciding whether to implement it.
Input	A change request describing the reason, affected feature, expected benefit, and urgency.
Initial review	The team checks whether the request is clear and whether it affects approved scope or acceptance criteria.
Impact analysis	The team estimates the effect on effort, schedule, architecture, security, testing, documentation, and other modules.
Decision	The team accepts, rejects, defers, or returns the request for clarification. The decision and its reason are recorded.
Implementation	For an approved request, update the backlog, WBS, design, code, tests, and relevant documentation.
Verification	Test the changed behavior and confirm that dependent features still work.
Exit criteria	The decision is recorded, approved changes are verified, and the affected project documents are updated.

9 Software Process Improvement

9.1 What is Process Improvement

Process improvement is the systematic effort to make work more reliable, efficient, and measurable. It uses evidence from reviews, testing, progress tracking, and stakeholder feedback to identify weaknesses and make controlled improvements.

For Grambandhan, process improvement can focus on reducing incomplete requirements, avoiding repeated defects, improving the reliability of transaction workflows, and making the interface easier to use for people with different levels of digital experience.

9.2 Six Sigma Methodology

Six Sigma is a data-oriented approach to reducing defects and variation in a process. Its DMAIC cycle consists of Define, Measure, Analyze, Improve, and Control.

9.2.1 Tools of Six Sigma

Useful tools include:

- **Process mapping:** Show the steps involved in a workflow such as project verification or payment confirmation.
- **Check sheets:** Record repeated defects or failed operations in a consistent format.
- **Pareto analysis:** Identify the small number of defect categories that cause most observed problems.
- **Cause-and-effect analysis:** Explore possible causes of a defect instead of correcting only its visible symptom.
- **Trend charts:** Track measures such as failed submissions or unresolved defects across test cycles.

The team should only calculate defect rates or claim a measured improvement when actual, consistently collected data is available.

9.3 ISO 9000 Quality Management Principles

The ISO 9000 family provides concepts and principles for quality management. These principles can guide Grambandhan's documentation, review, and improvement practices, but using them in a student project does not mean the project is ISO-certified.

9.3.1 Eight Core Quality Management Principles

1. **Customer focus:** Understand the needs of farmers, investors, buyers, and administrators.
2. **Leadership:** Keep project goals and responsibilities clear.
3. **Involvement of people:** Encourage team members to report issues and suggest improvements.
4. **Process approach:** Manage connected work as defined workflows rather than isolated activities.
5. **System approach to management:** Consider the effect of a change on the full platform.
6. **Continual improvement:** Use review and testing evidence to refine the system.
7. **Evidence-based decision-making:** Use recorded results instead of unsupported assumptions.
8. **Relationship management:** Consider dependencies on users, payment providers, hosting services, and other stakeholders.

10 Capability Maturity Model Integration (CMMI)

10.1 What is CMMI

Capability Maturity Model Integration (CMMI) is a framework for improving and assessing organizational processes. It encourages disciplined planning, monitoring, quality management, and continual improvement. It should not be treated as a certification claim for Grambandhan; the project can use selected practices without claiming a formal appraisal.

10.2 Objectives of the CMMI Framework

The main objectives relevant to Grambandhan are to make work more predictable, clarify responsibilities, improve the quality of deliverables, manage risks systematically, and use evidence to improve the development process.

10.3 Three Main Areas Addressed by CMMI

CMMI has been described through process-management, project-management, and engineering/support practices in various course materials. For Grambandhan, these can be understood practically as follows:

- **Process management:** Maintain consistent procedures for requirements, reviews, testing, and change control.
- **Project management:** Plan work, monitor progress, estimate effort, and respond to risks.
- **Engineering and support:** Design, implement, integrate, test, document, and maintain the platform.

10.4 CMMI Maturity-Level Structure

For this academic project, useful practices include a maintained WBS, documented requirements, an estimation baseline, a risk register, controlled changes, and recorded test results. The team should not claim that Grambandhan has achieved a formal CMMI maturity level unless an appropriate appraisal has been performed.

Table 13: CMMI Maturity Levels and Their Possible Relevance to Grambandhan

Level	Name	General Meaning
1	Initial	Work is mostly reactive and success may depend on individual effort.
2	Managed	Requirements, plans, responsibilities, and progress are tracked at project level.
3	Defined	Documented processes and standards are used consistently across the project.
4	Quantitatively Managed	Process performance is controlled using quantitative measures.
5	Optimizing	The organization uses systematic analysis and feedback to improve processes continuously.

11 Project Management Tools and Selection

11.1 What are Project Management Tools

Project management tools help teams plan tasks, assign responsibilities, monitor progress, record decisions, track risks, and maintain project documents. They provide a shared view of what is planned, what is in progress, and what remains unresolved.

11.2 Comparison of Project Management Tools

A suitable tool should match the team's size and working style. Grambandhan needs support for task tracking, code collaboration, issue recording, interface design, API testing, and report version control. Using too many disconnected tools can duplicate status information, so the team should decide where each type of information is maintained.

11.3 Selected Project Management Tools for Grambandhan

Table 14: Project Management Tools and Their Proposed Uses

Tool	Main Use	Possible Grambandhan Application
Trello	Visual task boards and checklists	Track project tasks through To Do, In Progress, Review, and Done.
Jira	Backlog, issue tracking, and sprint planning	Record feature requests, defects, priorities, and dependencies.

Tool	Main Use	Possible Grambandhan Application
GitHub Projects	Task boards connected to repository work	Link implementation tasks with issues, commits, and pull requests.
GitHub or GitLab	Version control and collaboration	Maintain code history and review changes before merging.
Figma	Interface prototyping	Prepare and review farmer, investor, buyer, and administrator workflows.
Postman	API testing and request collections	Test registration, project listing, investment, and payment-related endpoints.
Spreadsheet or shared document	Planning records and estimates	Maintain the WBS, effort baseline, risk register, and estimation forms.

For a small student team, a practical combination is a single task board, a shared version-control repository, an interface design tool, and a controlled folder for project documents. The team should select the tools it can maintain consistently rather than adopting every tool listed.

12 Software Review Techniques

12.1 What is a Review

A review is a systematic examination of a work product to identify defects, unclear requirements, inconsistencies, and opportunities for improvement before acceptance. Reviews can be applied to requirements, designs, code, test cases, project plans, and documentation.

12.2 Formal Inspection

An inspection is a structured review in which participants examine a work product against defined criteria and record defects for correction.

12.2.1 Roles in an Inspection Meeting

Typical roles include:

- **Author:** Prepares the work product and clarifies its intent.
- **Moderator:** Organizes the inspection and keeps the review focused.
- **Reader or presenter:** Guides participants through the material.
- **Reviewers:** Identify defects and check conformance to requirements.
- **Recorder:** Documents findings and action items.

In a small team, one person may perform more than one role, but the review should still be organized and its findings recorded.

12.2.2 Stages of the Inspection Process

The inspection process includes planning, individual preparation, a review meeting, correction of identified defects, and follow-up to verify that the issues were addressed. For Grambandhan, inspections may be useful for the WBS, payment workflow, access-control requirements, and final project report.

12.3 Desk Checking

Desk checking is a manual examination of logic, calculations, or a small algorithm without executing the full application. It can be used to review profit-sharing calculations, input validation conditions, project-status transitions, and risk-score rules.

12.4 Walkthroughs

In a walkthrough, the author explains a work product to other team members and receives questions and suggestions. Grambandhan walkthroughs can examine the registration process, project submission, investment flow, marketplace ordering, and payment-status screens. The goal is to reveal assumptions and usability problems early.

12.5 Code Reviews

Code reviews evaluate correctness, readability, security, maintainability, and test coverage. Changes involving authentication, role permissions, payment callbacks, user-submitted project data, or administrative verification should receive particular attention.

12.6 Collaborative Pair Programming

Pair programming involves two developers working together on the same task. One focuses on implementation while the other reviews the logic, asks questions, and considers edge cases. The roles should rotate so knowledge is shared across the team.

12.7 Participants in a Walkthrough

A Grambandhan walkthrough may include the author, a moderator, a technical reviewer, a user-interface reviewer, and a person considering the needs of farmers or investors. The participants should review both technical correctness and whether the workflow is understandable to its intended users.

12.8 Code Review Methods

The team may use pull-request reviews, pair reviews, asynchronous reviews, and checklist-based reviews. Pull-request reviews create a traceable record of comments and changes. Asynchronous reviews allow members to review at different times. Checklists help ensure that validation, permissions, error handling, logging, tests, and documentation are considered consistently.

12.9 Features of Pair Programming

Pair programming provides immediate feedback, shared understanding, early defect detection, and reduced dependence on one person's knowledge. It may be particularly useful for payment-status handling, role-based authorization, investment calculations, and integration between project modules.

12.10 Selected Review Techniques for Grambandhan

Grambandhan should use desk checking for small calculations and decision logic, walkthroughs for user workflows, code reviews for security-sensitive changes, and structured inspections for important documents and acceptance criteria. Review findings should be recorded, corrected, and rechecked before the related deliverable is accepted.

13 Conclusion and Future Work

13.1 Conclusion

This report establishes a project-management plan for Grambandhan, a proposed platform connecting rural project owners with investors and buyers. The methodology, WBS, effort baseline, risk plan, schedule, change control, process improvement, management tools, and review techniques provide a structured way to organize the project work.

The planning estimates and risk ratings are preliminary and should be validated against the detailed requirements and the team's actual estimation results. In particular, the available 385-hour task total should not be described as a completed Wideband Delphi

consensus until independent estimates and review rounds have been conducted. Similarly, FPA and COCOMO outputs require validated inputs before they can be used as reliable quantitative estimates.

13.2 Key Learnings

The planning exercise highlights that Grambandhan is not only a set of web pages. Its project listings, investment records, marketplace workflow, payment status, verification process, and risk-related information depend on one another. Clear task boundaries, careful handling of financial states, realistic estimates, and early testing are therefore important for maintaining trust and controlling scope.

13.3 Future Scope

Future development may include stronger project verification, improved transaction reconciliation, expanded marketplace features, better support for low-connectivity environments, more detailed project analytics, and carefully evaluated risk-assessment features. Additional capabilities should be introduced only after the core workflows have been tested and the team has the time and evidence needed to support them.

13.4 Final Remarks

Grambandhan can be developed in a more controlled way by maintaining its WBS, reviewing estimates as new evidence becomes available, monitoring risks, recording change decisions, and applying suitable review techniques. The next step is to confirm the remaining assumptions with the team and update this planning baseline as implementation and testing provide more reliable information.